Python 3.12 AsyncIO & High-Performance Coroutines

Presented By sam

Agenda

- • Introduction to AsyncIO
- • Understanding Coroutines
- • Writing Async Code
- • Exercise

Introduction to AsyncIO

- • AsyncIO for asynchronous programming
- • Non-blocking I/O operations
- • Example: Using asyncio.sleep()

```
// PYTHON 3.12.8
# Concept: Introduction to AsyncIO
message = "Introduction to AsyncIO Executed in Python 3.12"
print(message)
return message
asyncio.run(demonstrate_concept())
```

Understanding Coroutines

- • Coroutines are lightweight threads
- • Can be scheduled by the event loop
- • Example: Using asyncio.create_task()

```
// PYTHON 3.12.8
# Concept: Understanding Coroutines
message = "Understanding Coroutines Executed in Python 3.12"
print(message)
return message
asyncio.run(demonstrate_concept())
```

Writing Async Code

- • Using async and await keywords
- • Example: A simple async function that waits for
- • Example: A simple async function that

```
// PYTHON 3.12.8
# Concept: Writing Async Code
message = "Writing Async Code Executed in Python 3.12"
print(message)
return message
asyncio.run(demonstrate_concept())
```

Exercise: Simple Async Function

- • Write an async function that waits for user
- • Expected Output: 'Awaiting user input...', 'User input received'
- *Exercise Reference: exercise1*

```
// PYTHON 3.12.8
import asyncio
async def async_input():
    print('Awaiting user input...')
    await asyncio.sleep(1)
    print('User input received')
asyncio.run(async_input())
```

Summary

- • AsyncIO for non-blocking I/O operations
- • Coroutines are lightweight threads
- • Use async and await for asynchronous programming

Closing

- • Thank you for attending
- • Questions are welcome
- • References: Python 3.12.8 documentation